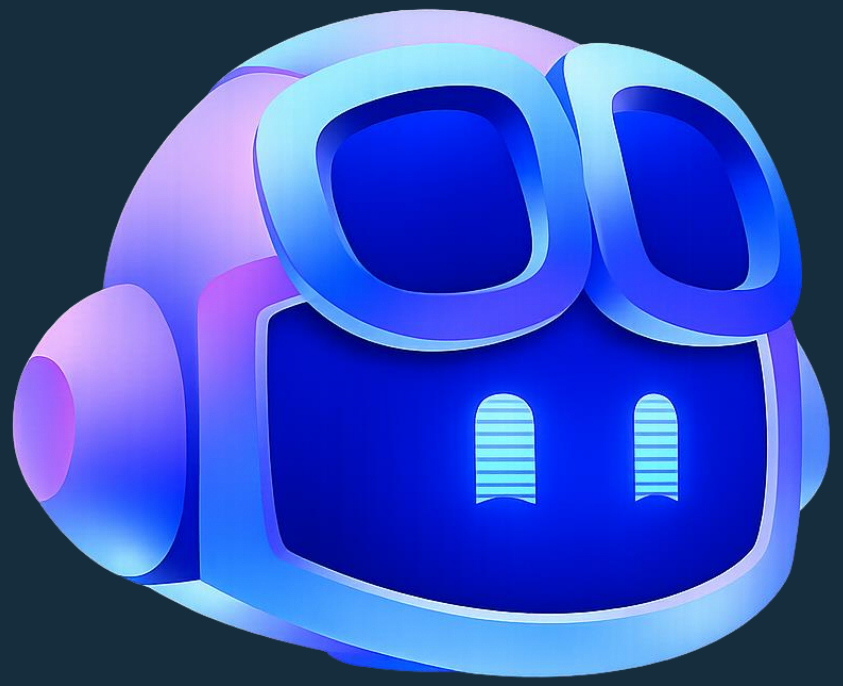
endava

AI-Native Foundations - GitHub Copilot (part 1)

Cómo usar comentarios, contexto, pruebas, explicación, refactorización y documentación para acelerar el trabajo sin perder criterio técnico.



Juan Felipe Castrillón
Internship Programme 2026-I

Agenda

01 Generación de código a partir de comentarios en lenguaje natural.

02 Autocompletado con conciencia del contexto del proyecto.

03 Ciclo de sugerencias inline y flujos de aceptación total o parcial.

04 Generación automática de unit tests y validación de casos borde.

05 Explicación y resumen de código para onboarding y mantenimiento.

06 Refactorización guiada por comentarios descriptivos.

07 Scaffolding de documentación con instrucciones reutilizables.

1. Generating code from natural-language comments

Convertir intención en una primera implementación útil

Idea principal

Un comentario claro define objetivo, entradas, restricciones y casos especiales.

Copilot lo usa para proponer boilerplate, algoritmos simples y validaciones iniciales.

La clave es explicitar qué debe respetarse.

Qué pedir

- Formato esperado.
- Reglas mínimas.
- Límites: no negativos, usar Decimal e ignorar nulos.

Comentario que guía a Copilot

```
# Calculate the final order total
# - sum price * qty
# - apply coupon_pct if present
# - never return a negative total
def calculate_total(items, coupon_pct=0):
```

Sugerencia inicial a revisar

```
def calculate_total(items, coupon_pct=0):
    subtotal = sum(item["price"] * item["qty"]
                   for item in items)
    discount = subtotal * (coupon_pct / 100)
    total = subtotal - discount
    return max(total, 0)
```

Lectura recomendada del ejemplo

La propuesta acelera el arranque. Todavía faltan tipos, redondeo, currency y edge cases.
Úsalo como primer borrador, no como código listo para producción.

2. Context-aware code completion

Copilot no completa “en vacío”: observa estilo, nombres, imports y archivos abiertos

Qué contexto aprovecha

- La función actual y el código cercano al cursor.
- Convenciones de nombres, estilo y estructura ya presentes.
- Otros archivos abiertos, como modelos o tests relacionados.

Implicación práctica

Antes de pedir una sugerencia, abre el archivo donde están los tipos, constantes o tests relevantes.
Mientras más coherente sea el contexto, mejor será la propuesta.

Código existente

```
from decimal import Decimal

class OrderService:
    def subtotal(self, items):
        return sum(Decimal(i.price) *
i.qty
                   for i in items)

    # add a tax_total method following
the same style
```

Lo que Copilot tiende a inferir

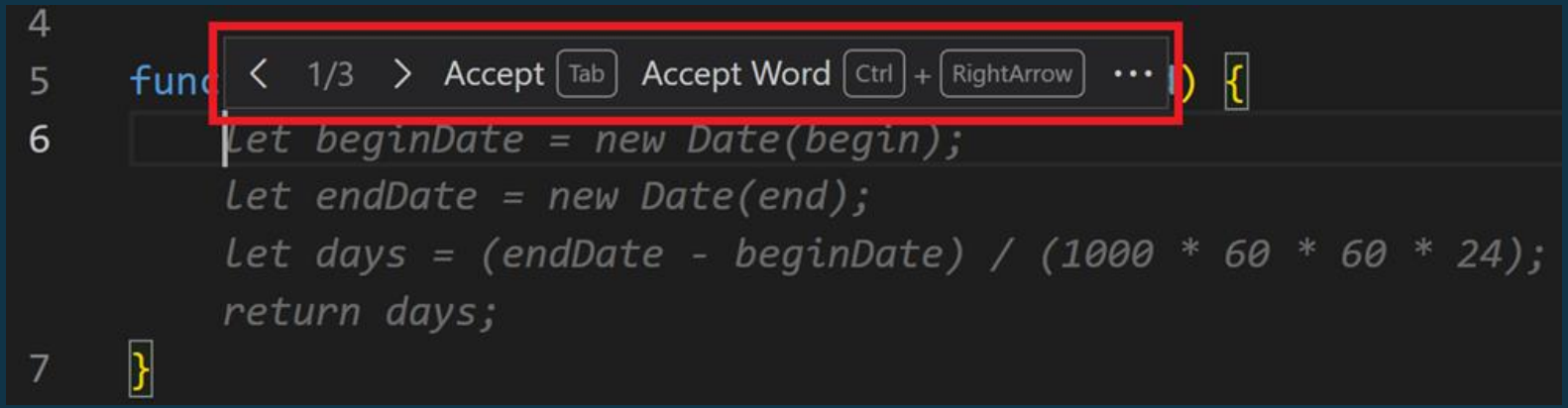
```
def tax_total(self, items, tax_rate):
    subtotal = self.subtotal(items)
    return subtotal *
Decimal(str(tax_rate))
```

3. Inline suggestion cycling y acceptance workflows

Aceptar todo no siempre es la mejor jugada

Flujo recomendado

1. Lee la sugerencia completa antes de aceptarla.
2. Acepta parcialmente si solo quieres la estructura o una línea.
3. Cicla entre alternativas cuando el enfoque no te convence.
4. Sigue escribiendo si ninguna propuesta sirve: tú mantienes el control.



Lo que conviene mostrar en vivo

- Aceptar la sugerencia completa.
- Aceptar solo una palabra o una línea.
- Cambiar a la siguiente alternativa.
- Ignorarla y continuar escribiendo para que Copilot se re-alinee.

4. Automatic generation of unit tests

Usar Copilot para cubrir comportamiento esperado, edge cases y regresiones

Prompt útil

“Generate jest unit tests for calculateTotal. Cover nominal case, 100% discount, empty items, and invalid quantity.”

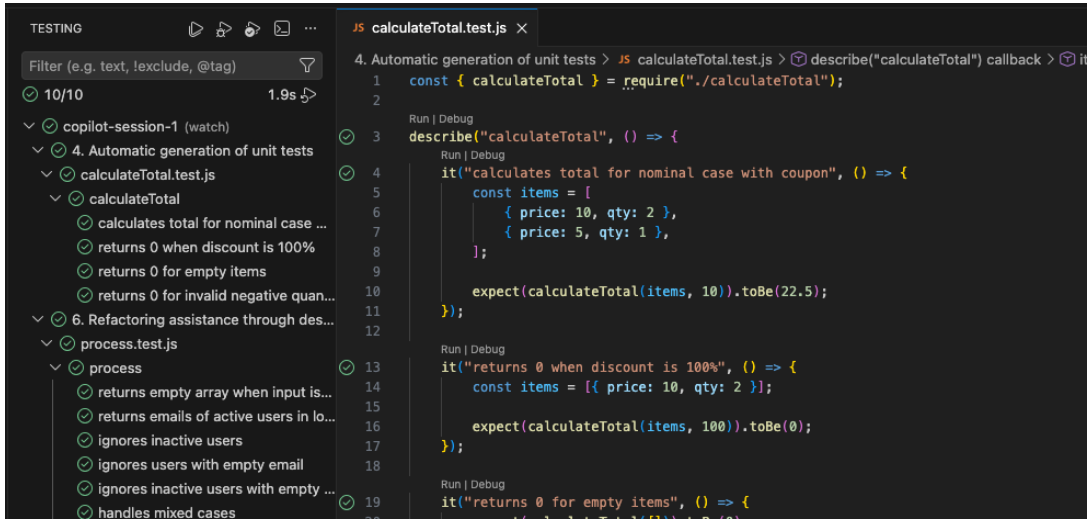
Resultado esperado para arrancar

```
it("calculates total for nominal case with coupon", () =>{
  const items = [
    { price: 10, qty: 2 },
    { price: 5, qty: 1 },
  ];

  expect(calculateTotal(items, 10)).toBe(22.5);
});


it("returns 0 for invalid negative quantity", () =>{
  const items = [{ price: 10, qty: -1 }];

  expect(calculateTotal(items)).toBe(0);
});
```



Qué validar antes de aceptar

- Que los asserts realmente cubran reglas de negocio.
- Que los nombres de las pruebas describan intención, no implementación.
- Que haya casos borde y no solo happy path.

5. Code explanation and summarization

Útil para onboarding, handover, revisión de incidentes y mantenimiento

Fragmento a explicar

```
def build_pricing_snapshot(order, rules,
clock):
    snapshot = []
    for item in order.items:
        active_rules = [r for r in rules
                        if r.applies_to(item,
clock)]
        for rule in active_rules:

snapshot.append(rule.calculate(item))
    return consolidate(snapshot)
```

Prompt ejemplo

“Explain this function in plain Spanish. Summarize inputs, outputs, side effects, and possible risks.”

Resumen que conviene esperar

Qué hace	Construye un snapshot de pricing recorriendo items y reglas activas.
Entradas	order, reglas de pricing y un reloj para evaluar vigencia temporal.
Salida	Una versión consolidada de ajustes, impuestos o descuentos aplicables.
Riesgos	Complejidad alta, reglas duplicadas, consolidación opaca y costo por loops anidados.

6. Refactoring assistance through descriptive comments

Pedir cambios de estructura sin reescribir desde cero

Antes

Función demasiado cargada

```
def checkout(order):
    # validates, prices, applies
tax,
    # applies coupon, persists
order,
    # emits event, logs metrics...
    ...
```

Comentario / prompt

“Refactor this function by extracting validation, pricing and persistence into helper methods. Keep behavior unchanged and improve readability.”

Después

Más legible y testeable

```
def checkout(order):
    validated = validate_order(order)
    priced = price_order(validated)
    saved = persist_order(priced)
    publish_checkout(saved)
    return saved
```

Regla de oro: refactorizar con red de seguridad. Idealmente: tests primero, luego refactor, luego diff y revisión.

7. Documentation scaffolding using prompt-driven instructions

Docstrings, README, API docs e instrucciones reutilizables

Escenarios típicos

- Generar o actualizar docstrings desde el código actual.
- Armar un README inicial con secciones estándar.
- Crear plantillas para documentación de endpoints o componentes.
- Usar instrucciones del repositorio para estandarizar tono y formato.

Prompts reutilizables

Ejemplos de instrucciones

```
/create-readme
/document-api endpoint_focus=GET /orders
Update the existing docstring to match the
current parameters
Add examples and error cases
```

Scaffold esperado

```
# Orders Service

## Overview
## Local setup
## Endpoints
## Example request/response
## Testing
## Operational notes

"""Calculate total order amount.
Args:
  items: list[OrderItem]
Returns:
  Decimal
Raises:
  ValueError
"""
```

Úsalo para arrancar rápido; luego edita lenguaje, ejemplos y detalles de negocio.

Demo

Buenas prácticas para una adopción sana

Maximizar productividad sin bajar el nivel de ingeniería

Copilot acelera la ejecución; la responsabilidad técnica sigue siendo del equipo.

Prompting

- Pide intención, restricciones y formato.
- Incluye stack, framework y casos borde.
- Usa ejemplos cortos cuando el dominio sea ambiguo.

Validación

- Lee siempre la sugerencia completa.
- Ejecuta tests o revisa diff.
- Verifica seguridad, performance y licencias.

Estandarización

- Define instrucciones del repo.
- Crea prompts reutilizables por equipo.
- Mantén ejemplos de referencia y snippets guía.

Riesgo

- Nunca pegues secretos o datos sensibles.
- No asumas que “si compila, está bien”.
- Evita aceptar cambios multiarchivo sin revisión.

Cierre

Tres ideas para que la audiencia se lleve a su siguiente sprint

1. Usa comentarios y prompts para expresar intención, no solo para pedir “código”.
2. Aprovecha contexto, aceptación parcial y tests para convertir sugerencias en cambios confiables.
3. Extiende el uso más allá del código: explicación, refactor y documentación también generan mucho valor.

Siguiente paso sugerido: correr un piloto corto en un repositorio real, con ejemplos de prompts, checklist de validación y medición simple de productividad.

Thank you!